Additional Bitwise Operators Problems

Filename: `No File Submission`

Instructions

This worksheet contains additional practice problems on bitwise operators. Try to complete each problem on your own before looking at the solutions. The solutions will be provided separately, so make sure to give each problem an honest attempt first. Checking the answer too early defeats the purpose of the practice. As you work, time yourself and do not spend more than 5 minutes on any single problem.

TODO 1

Complete the function that takes the number of bits (i.e., the length of the string) as input and returns how many binary strings of that length contain an odd number of 1s.

Example

Consider all binary strings of length 4. Since each bit can be either 0 or 1, there are $2^4 = 16$ possible strings. We want to count those that contain an odd number of 1s.
The valid 4-bit strings are:

$$0001,\ 0010,\ 0100,\ 1000,\ 0111,\ 1011,\ 1101,\ 1110.$$

Thus, the function should return 8.

```
int count_odd_ones(int bits) {






}
```

TODO 2

Complete the function that takes a positive integer N and returns the decimal value corresponding to the rightmost 1 bit in the binary representation of N.

In other words, starting from the rightmost digit (least significant bit), find the first occurrence of a 1 and return its value in decimal form.

Example

For example, given the input 12 (binary: 1100), the rightmost 1 corresponds to 0100, so the function should return 4.

```
int rightmost_one_bit_value(int N) {

}
```

TODO 3

Complete the function that takes a positive integer N and returns the decimal value corresponding to the leftmost 1 bit in the binary representation of N.

In other words, starting from the leftmost digit (most significant bit), find the first occurrence of a 1, and return its value in decimal form.

Assume that the integer is represented using 32 bits.

Example

For example, given the input 12 (binary: 1100), the leftmost 1 corresponds to 1000, so the function should return 8.

```
int leftmost_one_bit_value(int N) {

}
```

TODO 4

Complete the function that takes two positive integers: a number N and a pattern P, both given in decimal form. The function should return `1` if the rightmost bits of N, with the same length as the binary representation of P, exactly match the binary pattern represented by P, and `0` otherwise.

Example

For example, let $N = 13$ (binary: `1101`) and $P = 5$ (binary: `101`). The rightmost 3 bits of N are `101`, which match the pattern, so the function should return `1`.

Important: The number of bits to compare is determined by the length of P's binary representation (ignoring leading zeros).

```
int match_pattern(int N, int pattern) {

}
```